AMRITA VISHWA VIDYAPEETHAM | School of Engineering
Chennai Campus

Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

KANTAMRAJU SREEJA

CH.SC.U4CSE24255

WORKSHEET-3

WORKSHEET- 3

EX.NO : 3

DATE : 10-10-26

TITLE : Quantum Communication Protocols: Superdense Coding and Quantum Teleportation Using Qiskit

AIM:

To implement the superdense coding and quantum teleportation protocols in Qiskit using a shared Bell pair, and to verify that two classical bits can be sent with one qubit and that the state of a qubit can be transferred with two classical bits.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit 2.x and Qiskit Aer simulator (`pip install qiskit qiskit-aer`)
- Matplotlib and pylatexenc (for circuit diagrams and histograms)
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

Both protocols in this experiment use one shared pair of entangled qubits, called a Bell pair, prepared with a Hadamard gate and a CNOT gate. Alice holds one qubit of the pair and Bob holds the other.

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Superdense coding sends two classical bits by transmitting one qubit. Alice applies one of four operations to her half of the pair, chosen by the two bits b_1b_0 she wants to send: an X gate if $b_0 = 1$, followed by a Z gate if $b_1 = 1$. The messages 00, 01, 10 and 11 therefore use I, X, Z and ZX. These turn the pair into four different Bell states, which can be told apart with certainty.

Alice then sends her qubit to Bob. Bob applies a CNOT gate with Alice's qubit as the control, applies a Hadamard gate to Alice's qubit, and measures both qubits to read the two bits.

Someone who intercepts only the travelling qubit learns nothing: measured on its own, it gives 0 or 1 with equal probability whatever the message is.

Quantum teleportation does the reverse: it transfers the state of one qubit by sending two classical bits. Alice holds a qubit in an unknown state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ and her half of a Bell pair. She applies a CNOT gate from the unknown qubit to her half of the pair, applies a Hadamard gate to the unknown qubit, and measures both. Just before the measurement the three qubits are in the state

$$\frac{1}{2}[\,|00\rangle(\alpha|0\rangle + \beta|1\rangle) + |01\rangle(\alpha|1\rangle + \beta|0\rangle) + |10\rangle(\alpha|0\rangle - \beta|1\rangle) + |11\rangle(\alpha|1\rangle - \beta|0\rangle)]$$

Here the first two positions are Alice's qubits (the unknown qubit, then her half of the pair) and the third is Bob's qubit. Alice's measurement gives two bits m_1m_2 and leaves Bob's qubit in one of the four states in the brackets. Alice sends the two bits to Bob, who applies an X gate if $m_2 = 1$ and then a Z gate if $m_1 = 1$. His qubit is then exactly $|\psi\rangle$.

Three facts follow. The state is moved, not copied: Alice's measurement destroys her original, so the no-cloning theorem holds. Nothing travels faster than light: without the two classical bits, Bob's qubit gives random results. Only the state is transferred; no matter moves.

The two protocols are mirror images of each other. One shared Bell pair and one transmitted qubit carry two classical bits. One shared Bell pair and two transmitted classical bits carry one qubit.

In Qiskit, a gate that depends on a measurement result is written inside `with qc.if_test((creg, 1)):`, and such circuits are run on `AerSimulator`. Use a separate classical register for each measured bit so that each condition is easy to write. Qiskit prints qubit 0 as the rightmost bit.

PROCEDURE:

1. Install Qiskit and the Aer simulator with `pip install qiskit qiskit-aer matplotlib pylatexenc`, and print the installed version with `qiskit._version_`.

2. Open Jupyter Notebook or Visual Studio Code.

3. Import `QuantumCircuit` and `transpile` from `qiskit`, `Statevector` and `Operator` from `qiskit.quantum_info`, `AerSimulator` from `qiskit_aer` and `plot_histogram` from `qiskit.visualization`. Import `QuantumRegister` and `ClassicalRegister` as well.

4. Create the Bell pair with a Hadamard gate and a CNOT gate, and decide which qubit belongs to Alice and which to Bob.

5. Superdense coding: apply Alice's encoding gates for the chosen message, then Bob's CNOT and Hadamard gates, and measure both qubits.

6. Teleportation: prepare the state to be sent on a third qubit, apply Alice's CNOT and Hadamard gates, and measure her two qubits into separate classical registers.

7. Apply Bob's corrections inside `with qc.if_test(...)` blocks: X if the second measured bit is 1, then Z if the first measured bit is 1.

8. Draw the circuit with `qc.draw("mpl")` before running it. Use barriers to separate the stages of the protocol.

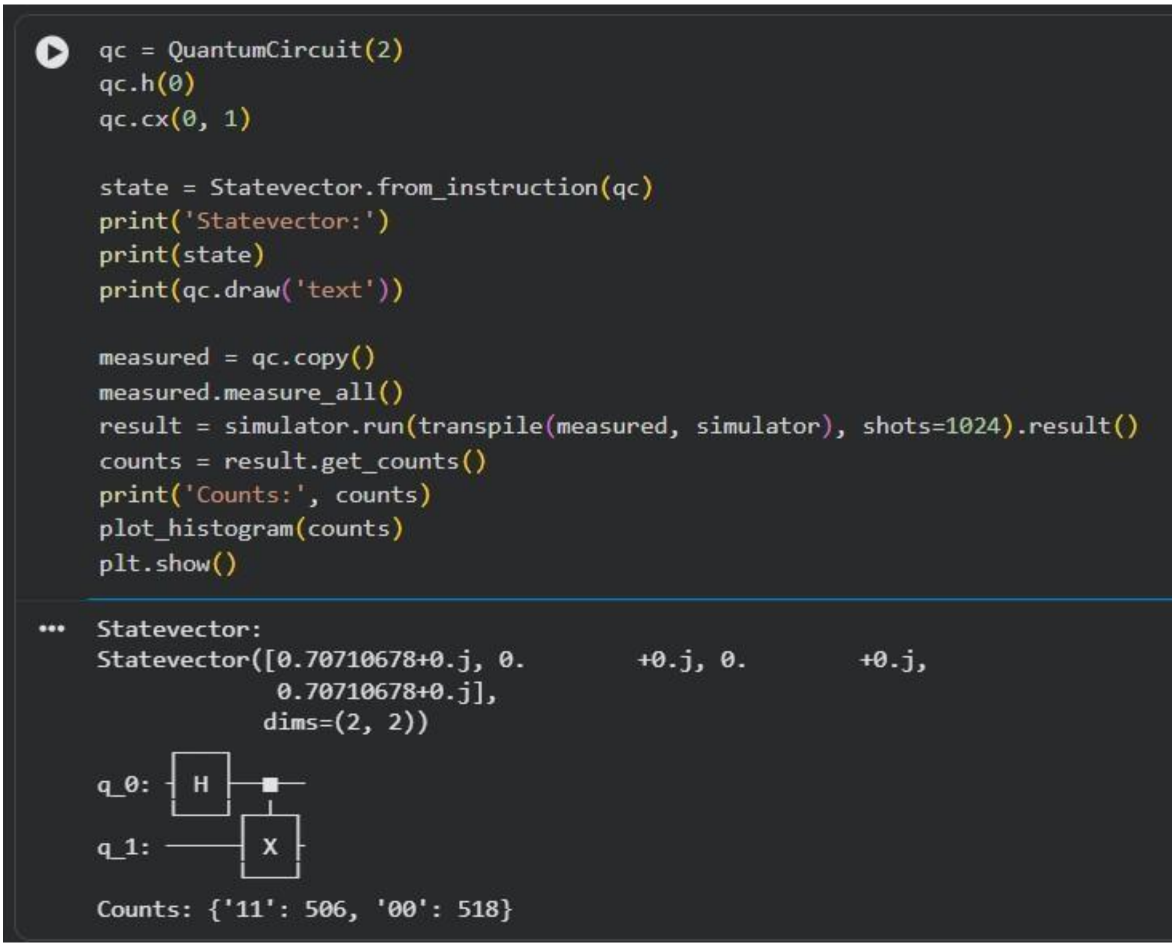
9. Run the circuit on `AerSimulator` with 1024 shots and plot the histogram of the relevant bits.

10. Compare the result with the hand calculation. Remember that Qiskit prints qubit 0 as the rightmost bit. Record the code, the circuit diagram, the output and a one-line inference under each program.

PROGRAM:

Program 1: Prepare a Bell pair shared between Alice and Bob. Display the statevector and the histogram for 1024 shots.

OUTPUT:

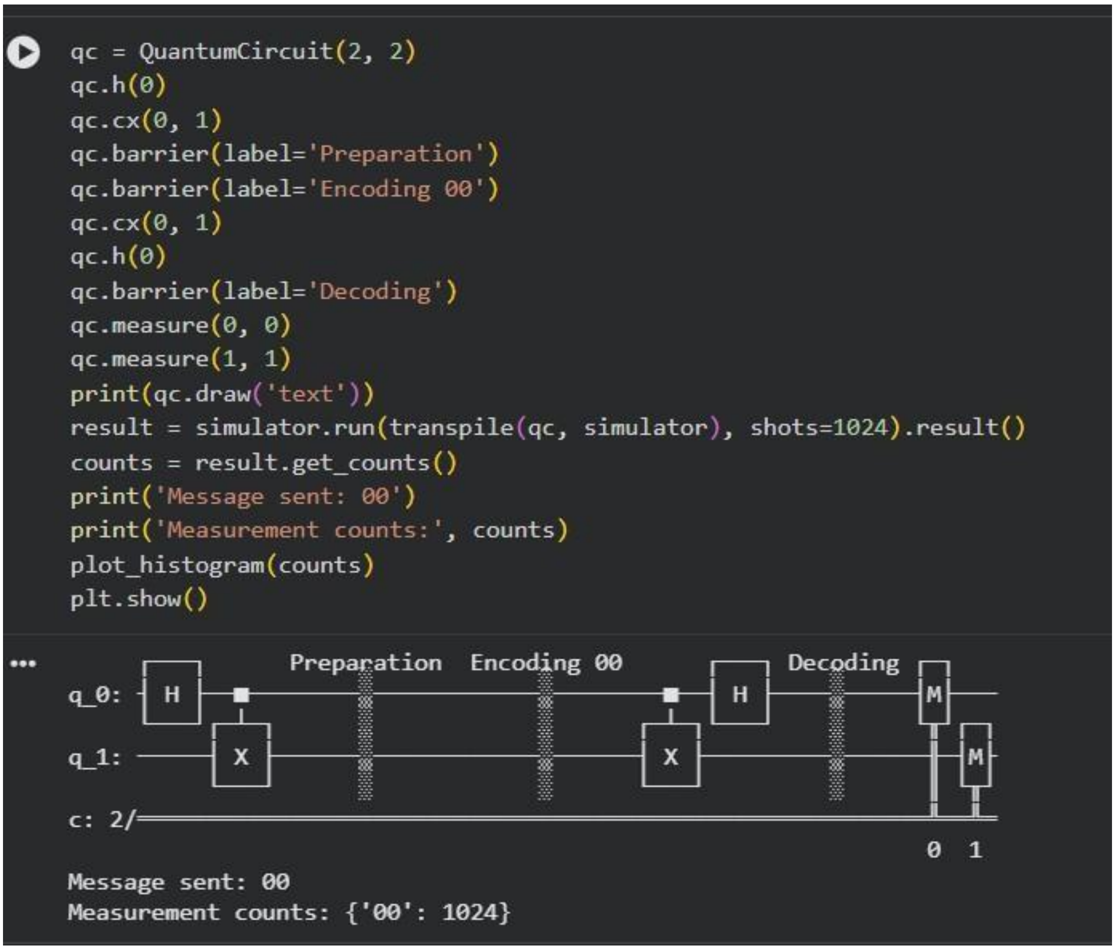


```
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

state = Statevector.from_instruction(qc)
print('Statevector:')
print(state)
print(qc.draw('text'))

measured = qc.copy()
measured.measure_all()
result = simulator.run(transpile(measured, simulator), shots=1024).result()
counts = result.get_counts()
print('Counts:', counts)
plot_histogram(counts)
plt.show()
```

```
Statevector:
Statevector([0.70710678+0.j, 0.        +0.j, 0.        +0.j,
             0.70710678+0.j],
            dims=(2, 2))
     ┌───┐
q_0: ┤ H ├──■──
     └───┘┌─┴─┐
q_1: ─────┤ X ├
          └───┘
Counts: {'11': 506, '00': 518}
```

Program 2: Implement superdense coding for the message 00. Draw the complete circuit with barriers between the preparation, encoding and decoding stages, and show the measurement result.

OUTPUT:

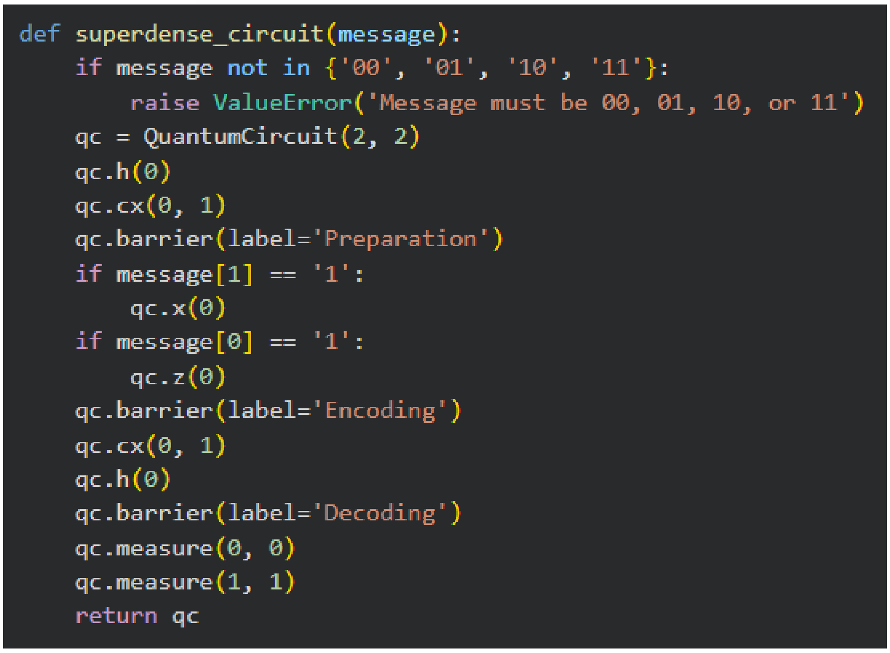


```python
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.barrier(label='Preparation')
qc.barrier(label='Encoding 00')
qc.cx(0, 1)
qc.h(0)
qc.barrier(label='Decoding')
qc.measure(0, 0)
qc.measure(1, 1)
print(qc.draw('text'))
result = simulator.run(transpile(qc, simulator), shots=1024).result()
counts = result.get_counts()
print('Message sent: 00')
print('Measurement counts:', counts)
plot_histogram(counts)
plt.show()
```

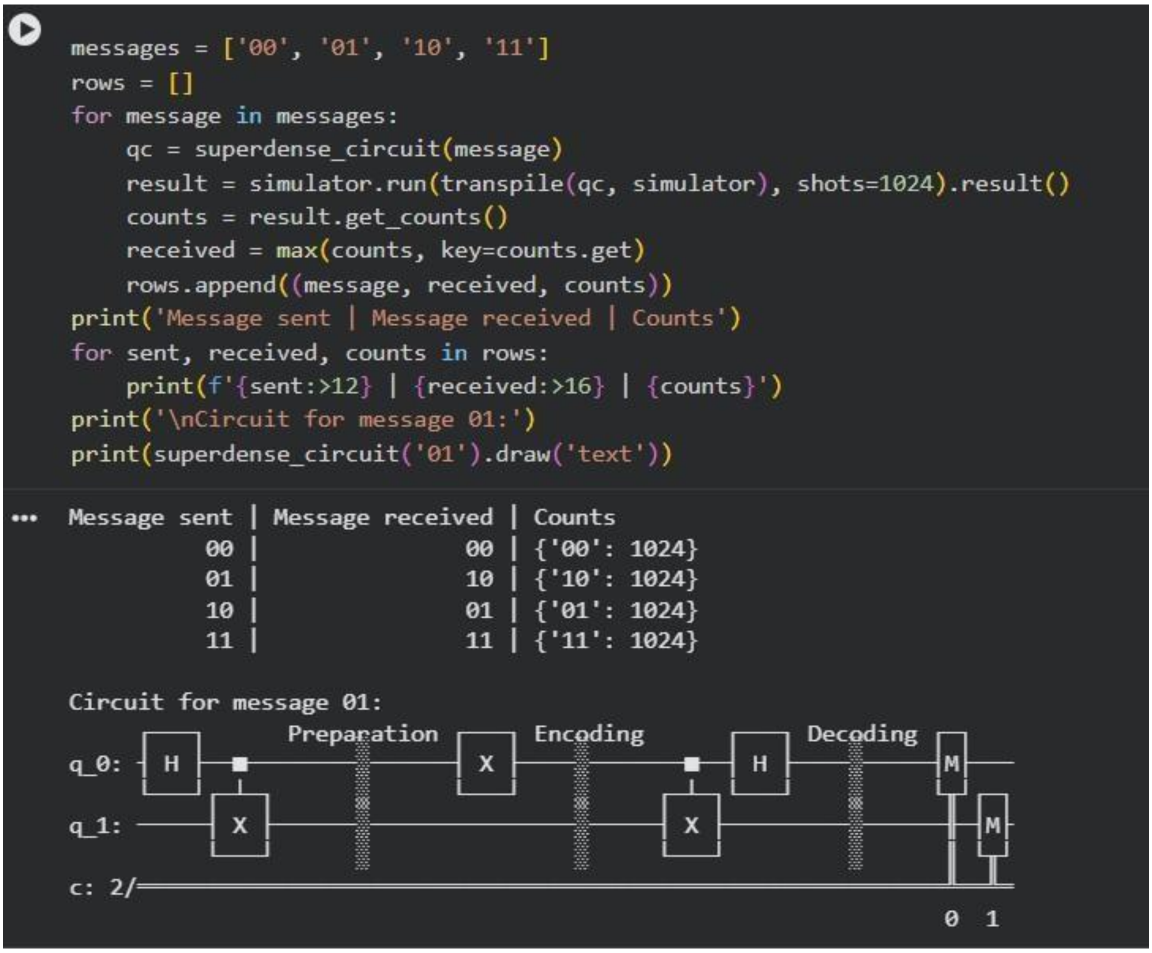
```
     ┌───┐     Preparation  Encoding 00      ┌───┐ Decoding ┌─┐   
q_0: ┤ H ├──■───────░────────────░────────■──┤ H ├────░─────┤M├───
     └───┘┌─┴─┐     ░            ░      ┌─┴─┐└───┘    ░     └╥┘┌─┐
q_1: ─────┤ X ├─────░────────────░──────┤ X ├─────────░──────╫─┤M├
          └───┘     ░            ░      └───┘         ░      ║ └╥┘
c: 2/════════════════════════════════════════════════════════╩══╩═
                                                             0  1 

Message sent: 00
Measurement counts: {'00': 1024}
```

Program 3: Write a function that builds the superdense coding circuit for any two-bit message. Run it for 00, 01, 10 and 11 and tabulate the message sent against the message received.

OUTPUT:



```python
def superdense_circuit(message):
    if message not in {'00', '01', '10', '11'}:
        raise ValueError('Message must be 00, 01, 10, or 11')
    qc = QuantumCircuit(2, 2)
    qc.h(0)
    qc.cx(0, 1)
    qc.barrier(label='Preparation')
    if message[1] == '1':
        qc.x(0)
    if message[0] == '1':
        qc.z(0)
    qc.barrier(label='Encoding')
    qc.cx(0, 1)
    qc.h(0)
    qc.barrier(label='Decoding')
    qc.measure(0, 0)
    qc.measure(1, 1)
    return qc
```

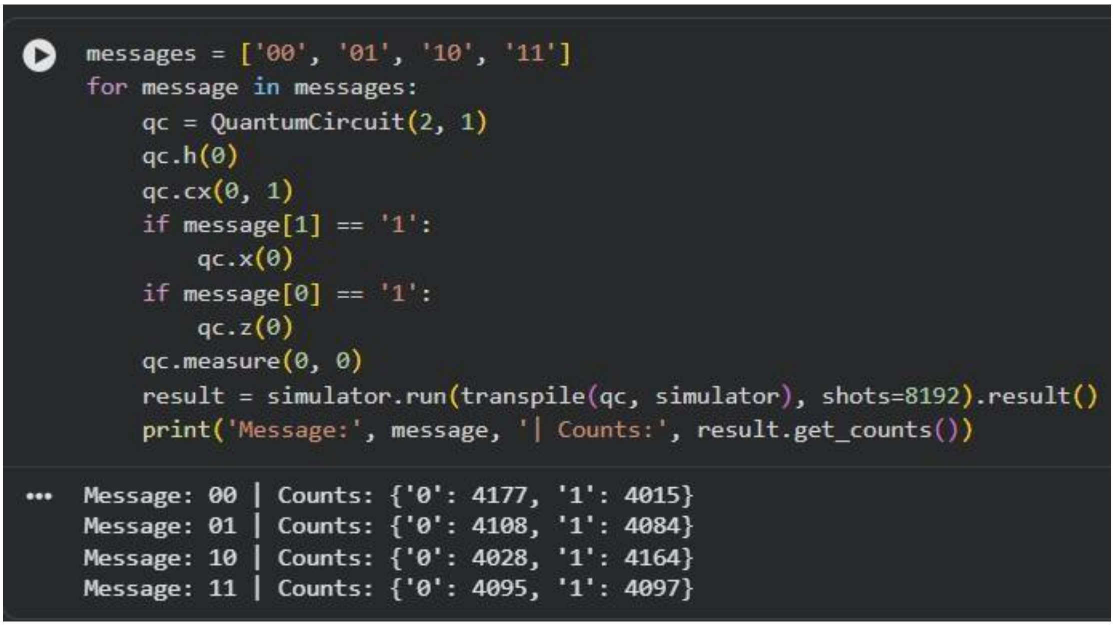


```python
messages = ['00', '01', '10', '11']
rows = []
for message in messages:
    qc = superdense_circuit(message)
    result = simulator.run(transpile(qc, simulator), shots=1024).result()
    counts = result.get_counts()
    received = max(counts, key=counts.get)
    rows.append((message, received, counts))
print('Message sent | Message received | Counts')
for sent, received, counts in rows:
    print(f'{sent:>12} | {received:>16} | {counts}')
print('\nCircuit for message 01:')
print(superdense_circuit('01').draw('text'))
```

```
Message sent | Message received | Counts
          00 |               00 | {'00': 1024}
          01 |               10 | {'10': 1024}
          10 |               01 | {'01': 1024}
          11 |               11 | {'11': 1024}

Circuit for message 01:
     ┌───┐      Preparation ┌───┐ Encoding      ┌───┐ Decoding ┌─┐   
q_0: ┤ H ├──■────────░──────┤ X ├────░─────■──┤ H ├────░─────┤M├───
     └───┘┌─┴─┐      ░      └───┘    ░   ┌─┴─┐└───┘    ░     └╥┘┌─┐
q_1: ─────┤ X ├──────░───────────────░───┤ X ├─────────░──────╫─┤M├
          └───┘      ░               ░   └───┘         ░      ║ └╥┘
c: 2/═════════════════════════════════════════════════════════╩══╩═
                                                              0  1 
```

Program 4: For each of the four messages, measure only the qubit that Alice sends, without Bob's decoding. Show that the result is 0 or 1 with equal probability in every case, and explain what this means for an eavesdropper.

OUTPUT:

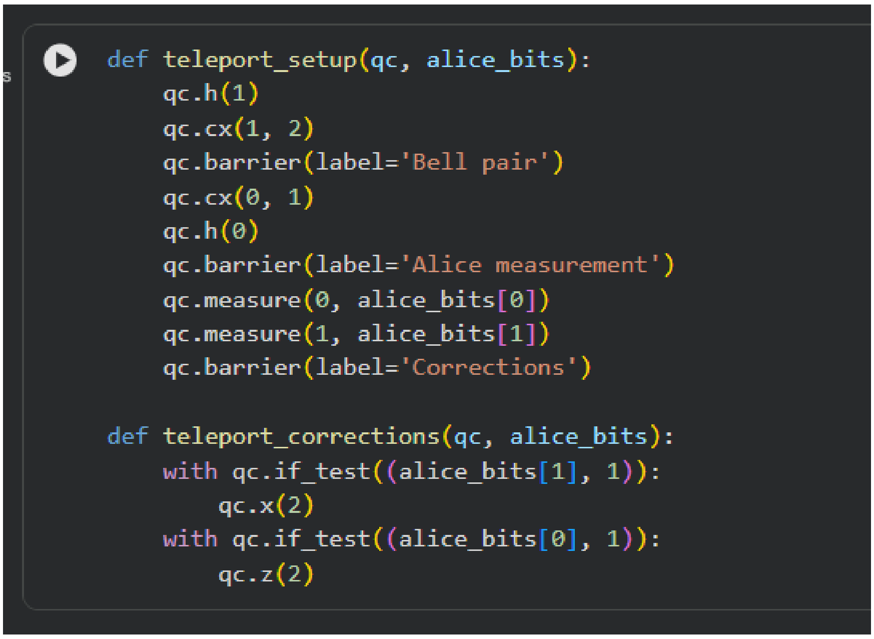


```
messages = ['00', '01', '10', '11']
for message in messages:
    qc = QuantumCircuit(2, 1)
    qc.h(0)
    qc.cx(0, 1)
    if message[1] == '1':
        qc.x(0)
    if message[0] == '1':
        qc.z(0)
    qc.measure(0, 0)
    result = simulator.run(transpile(qc, simulator), shots=8192).result()
    print('Message:', message, '| Counts:', result.get_counts())
```

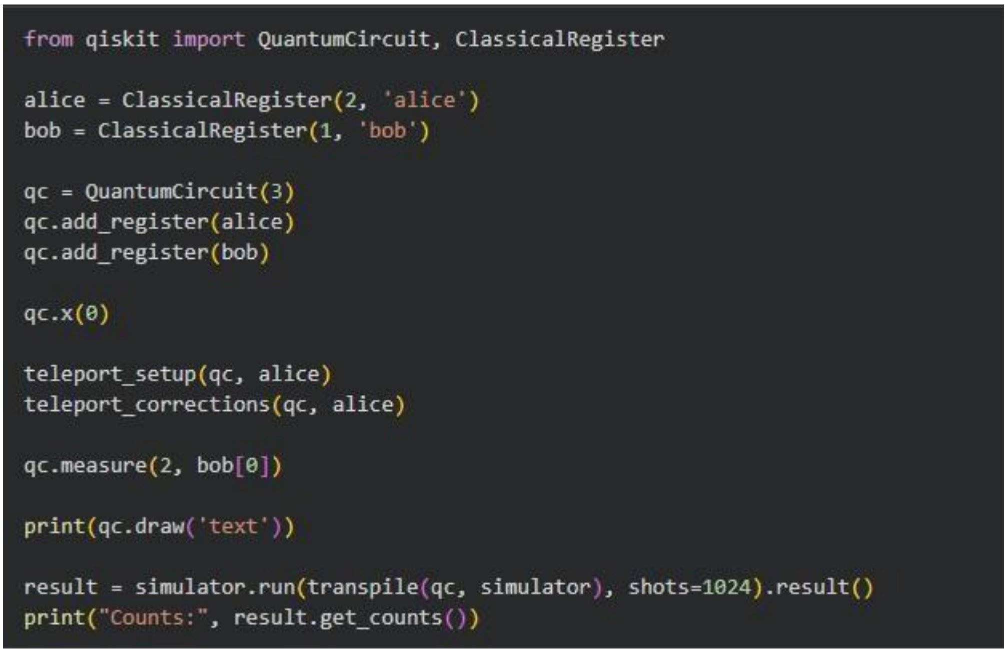
```
Message: 00 | Counts: {'0': 4177, '1': 4015}
Message: 01 | Counts: {'0': 4108, '1': 4084}
Message: 10 | Counts: {'0': 4028, '1': 4164}
Message: 11 | Counts: {'0': 4095, '1': 4097}
```

Program 5: Implement quantum teleportation of the state |1⟩ using three qubits and classically controlled corrections. Measure Bob's qubit and show that it is always 1.

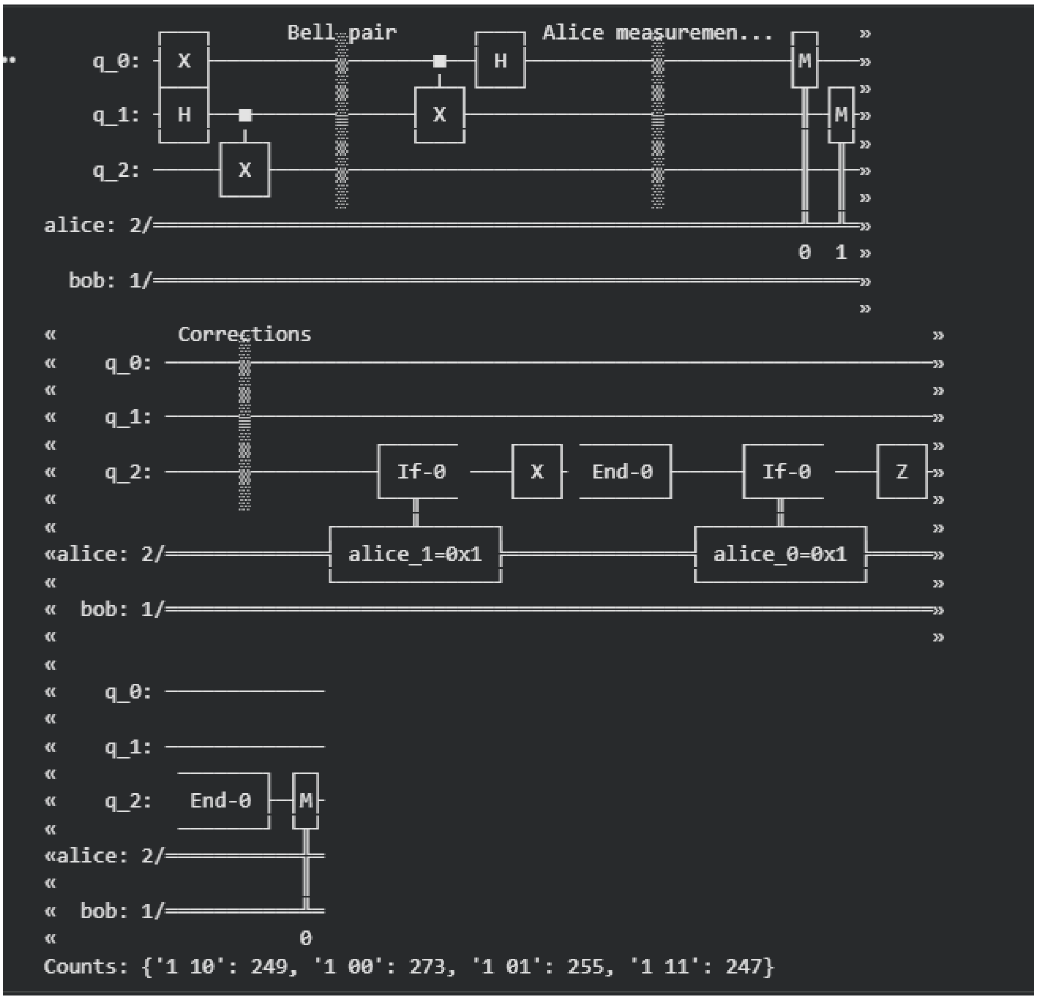
OUTPUT:



```
def teleport_setup(qc, alice_bits):
    qc.h(1)
    qc.cx(1, 2)
    qc.barrier(label='Bell pair')
    qc.cx(0, 1)
    qc.h(0)
    qc.barrier(label='Alice measurement')
    qc.measure(0, alice_bits[0])
    qc.measure(1, alice_bits[1])
    qc.barrier(label='Corrections')

def teleport_corrections(qc, alice_bits):
    with qc.if_test((alice_bits[1], 1)):
        qc.x(2)
    with qc.if_test((alice_bits[0], 1)):
        qc.z(2)
```



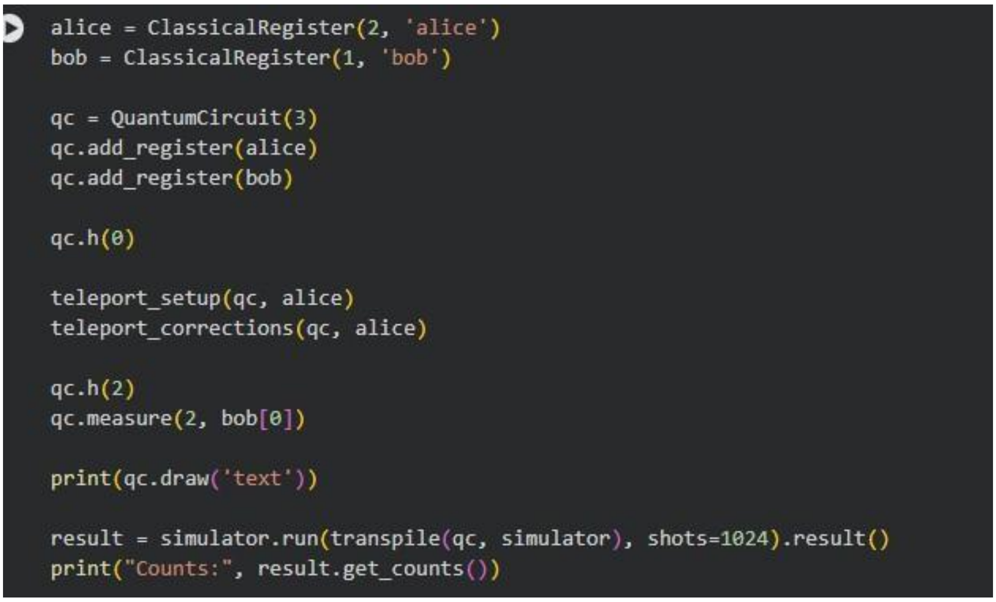
```python
from qiskit import QuantumCircuit, ClassicalRegister

alice = ClassicalRegister(2, 'alice')
bob = ClassicalRegister(1, 'bob')

qc = QuantumCircuit(3)
qc.add_register(alice)
qc.add_register(bob)

qc.x(0)

teleport_setup(qc, alice)
teleport_corrections(qc, alice)

qc.measure(2, bob[0])

print(qc.draw('text'))

result = simulator.run(transpile(qc, simulator), shots=1024).result()
print("Counts:", result.get_counts())
```



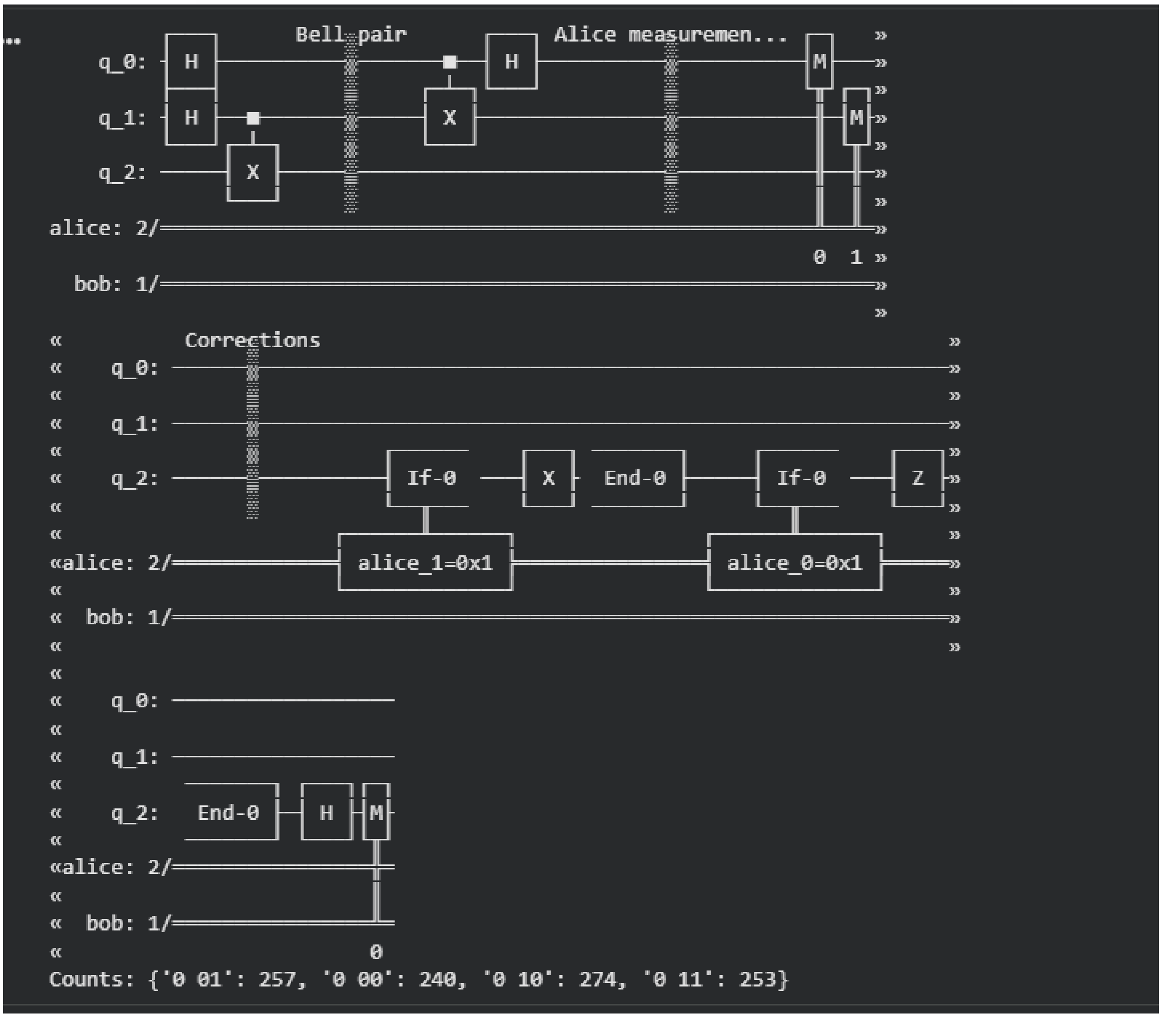
```
         ┌───┐       Bell pair      ┌───┐ Alice measuremen...  ┌─┐   »
    q_0: ┤ X ├───────────░───────■──┤ H ├──────────░───────────┤M├───»
         ├───┤           ░     ┌─┴─┐└───┘          ░           └╥┘┌─┐»
    q_1: ┤ H ├──■────────░─────┤ X ├───────────────░────────────╫─┤M├»
         └───┘┌─┴─┐      ░     └───┘               ░            ║ └╥┘»
    q_2: ─────┤ X ├──────░─────────────────────────░────────────╫──╫─»
              └───┘      ░                         ░            ║  ║ »
alice: 2/═══════════════════════════════════════════════════════╩══╩═»
                                                                0  1 »
  bob: 1/════════════════════════════════════════════════════════════»
                                                                     »
«          Corrections                                                     »
«    q_0: ──────░──────────────────────────────────────────────────────────»
«               ░                                                          »
«    q_1: ──────░──────────────────────────────────────────────────────────»
«               ░           ┌──────    ┌───┐ ────────┐     ┌──────    ┌───┐»
«    q_2: ──────░───────────┤ If-0  ───┤ X ├  End-0  ├─────┤ If-0  ───┤ Z ├»
«               ░           └──╥───    └───┘ ────────┘     └──╥───    └───┘»
«                       ┌──────╨──────┐                ┌──────╨──────┐     »
«alice: 2/══════════════╡ alice_1=0x1 ╞════════════════╡ alice_0=0x1 ╞═════»
«                       └─────────────┘                └─────────────┘     »
«  bob: 1/═════════════════════════════════════════════════════════════════»
«                                                                          »
«
«    q_0: ───────────────
«
«    q_1: ───────────────
«          ───────┐  ┌─┐
«    q_2:   End-0 ├──┤M├─
«          ───────┘  └╥┘
«alice: 2/════════════╫══
«                     ║
«  bob: 1/════════════╩══
«                     0
Counts: {'1 10': 249, '1 00': 273, '1 01': 255, '1 11': 247}
```

Program 6: Teleport the state |+⟩. Apply a Hadamard gate to Bob's qubit before measuring it and show that the result is always 0.

OUTPUT:



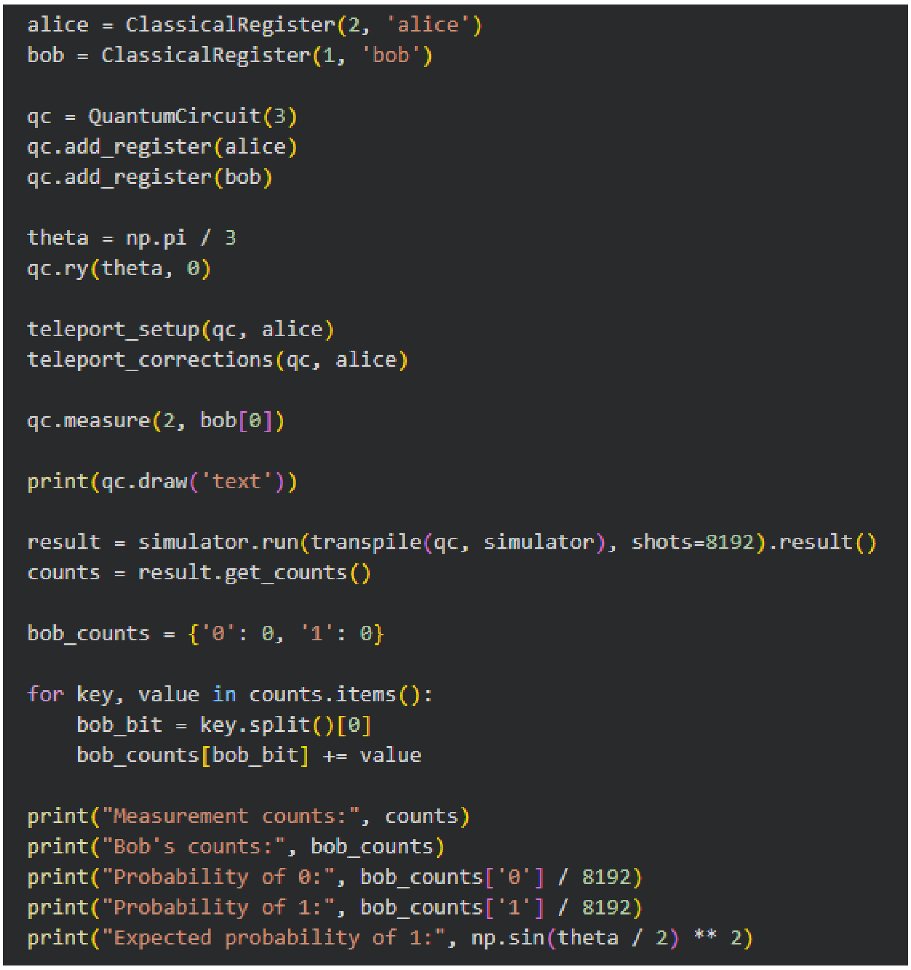
```
alice = ClassicalRegister(2, 'alice')
bob = ClassicalRegister(1, 'bob')

qc = QuantumCircuit(3)
qc.add_register(alice)
qc.add_register(bob)

qc.h(0)

teleport_setup(qc, alice)
teleport_corrections(qc, alice)

qc.h(2)
qc.measure(2, bob[0])

print(qc.draw('text'))

result = simulator.run(transpile(qc, simulator), shots=1024).result()
print("Counts:", result.get_counts())
```



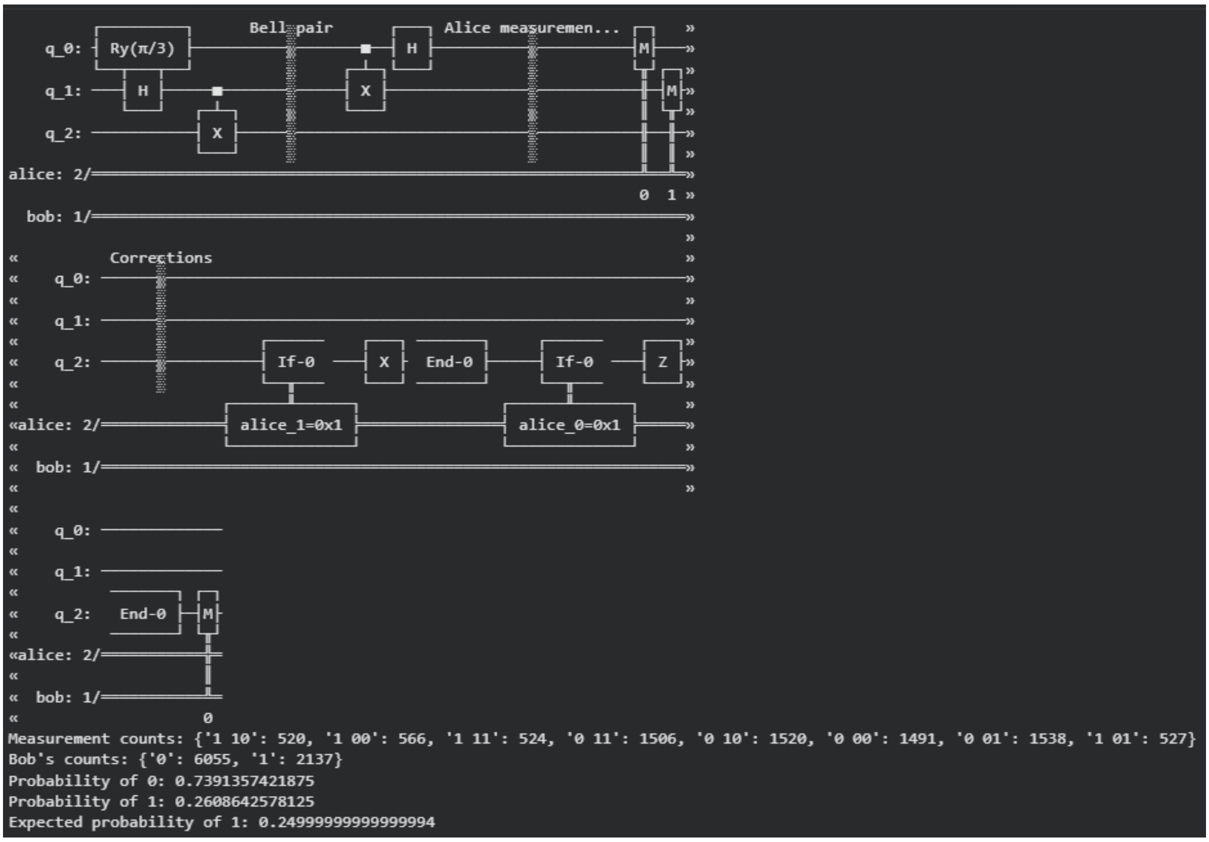
```
•••
          ┌───┐          Bell pair      ┌───┐ Alice measuremen... ┌─┐      »
     q_0: ┤ H ├─────────────░──────■────┤ H ├───────────░─────────┤M├──────»
          ├───┤             ░    ┌─┴─┐  └───┘           ░         └╥┘┌─┐   »
     q_1: ┤ H ├──■──────────░────┤ X ├──────────────────░──────────╫─┤M├───»
          └───┘┌─┴─┐        ░    └───┘                  ░          ║ └╥┘   »
     q_2: ─────┤ X ├────────░───────────────────────────░──────────╫──╫────»
               └───┘        ░                           ░          ║  ║    »
 alice: 2/═════════════════════════════════════════════════════════╩══╩════»
                                                                   0  1    »
   bob: 1/═════════════════════════════════════════════════════════════════»
                                                                           »
«          Corrections                                                        »
«    q_0: ──────░─────────────────────────────────────────────────────────────»
«               ░                                                             »
«    q_1: ──────░─────────────────────────────────────────────────────────────»
«               ░              ┌──────    ┌───┐ ───────┐      ┌──────    ┌───┐»
«    q_2: ──────░──────────────┤ If-0  ───┤ X ├─ End-0 ├──────┤ If-0  ───┤ Z ├»
«               ░              └──╥───    └───┘ ───────┘      └──╥───    └───┘»
«                              ┌──╨────────┐                ┌───╨────────┐    »
«alice: 2/═════════════════════╡ alice_1=0x1 ╞════════════════╡ alice_0=0x1 ╞══»
«                              └───────────┘                └────────────┘    »
«  bob: 1/════════════════════════════════════════════════════════════════════»
«                                                                             »
«
«    q_0: ─────────────────
«
«    q_1: ─────────────────
«         ───────┐┌───┐┌─┐
«    q_2:  End-0 ├┤ H ├┤M├
«         ───────┘└───┘└╥┘
«alice: 2/══════════════╬═
«                       ║
«  bob: 1/══════════════╩═
«                       0
Counts: {'0 01': 257, '0 00': 240, '0 10': 274, '0 11': 253}
```

Program 7: Teleport the state $R_y(\theta)|0\rangle$ with $\theta = \pi/3$. Show that Bob's qubit measures 1 with probability $\sin^2(\theta/2) = 0.25$, using at least 8192 shots.

OUTPUT:



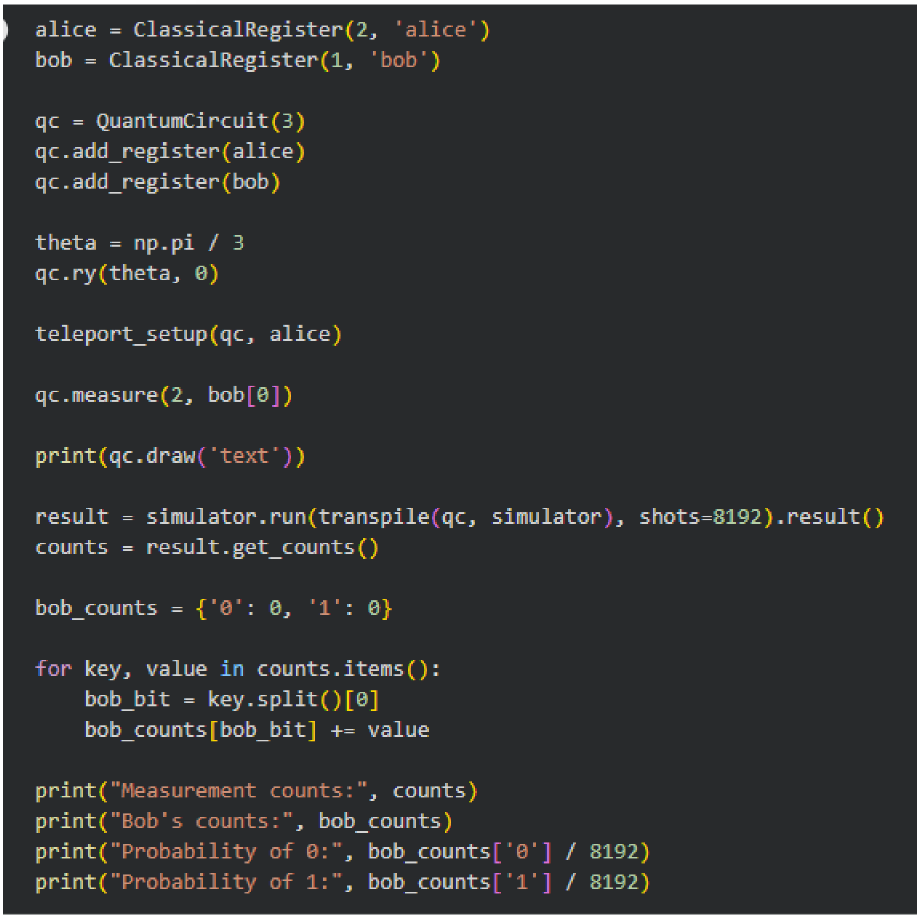
```
alice = ClassicalRegister(2, 'alice')
bob = ClassicalRegister(1, 'bob')

qc = QuantumCircuit(3)
qc.add_register(alice)
qc.add_register(bob)

theta = np.pi / 3
qc.ry(theta, 0)

teleport_setup(qc, alice)
teleport_corrections(qc, alice)

qc.measure(2, bob[0])

print(qc.draw('text'))

result = simulator.run(transpile(qc, simulator), shots=8192).result()
counts = result.get_counts()

bob_counts = {'0': 0, '1': 0}

for key, value in counts.items():
    bob_bit = key.split()[0]
    bob_counts[bob_bit] += value

print("Measurement counts:", counts)
print("Bob's counts:", bob_counts)
print("Probability of 0:", bob_counts['0'] / 8192)
print("Probability of 1:", bob_counts['1'] / 8192)
print("Expected probability of 1:", np.sin(theta / 2) ** 2)
```



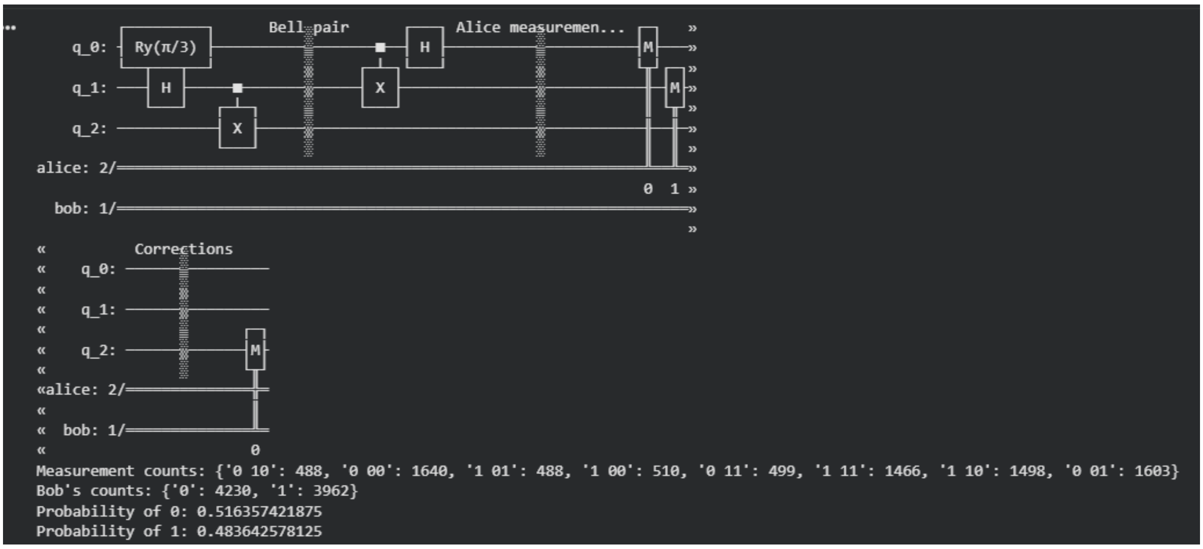
```
        ┌─────────┐          Bell pair      ┌───┐ Alice measuremen... ┌─┐    »
   q_0: ┤ Ry(π/3) ├──────────────░──────■───┤ H ├──────────░──────────┤M├────»
        └──┬───┬──┘              ░    ┌─┴─┐ └───┘          ░          └╥┘┌─┐ »
   q_1: ───┤ H ├──────■──────────░────┤ X ├────────────────░───────────╫─┤M├─»
           └───┘    ┌─┴─┐        ░    └───┘                ░           ║ └╥┘ »
   q_2: ────────────┤ X ├────────░─────────────────────────░───────────╫──╫──»
                    └───┘        ░                         ░           ║  ║  »
alice: 2/══════════════════════════════════════════════════════════════╩══╩══»
                                                                       0  1  »
  bob: 1/════════════════════════════════════════════════════════════════════»
                                                                             »
«         Corrections                                                        »
«    q_0: ─────░─────────────────────────────────────────────────────────────»
«              ░                                                             »
«    q_1: ─────░─────────────────────────────────────────────────────────────»
«              ░           ┌──────     ┌───┐ ───────┐        ┌──────     ┌───┐»
«    q_2: ─────░───────────┤ If-0  ────┤ X ├─ End-0 ├────────┤ If-0  ────┤ Z ├»
«              ░           └──╥───     └───┘ ───────┘        └──╥───     └───┘»
«                       ┌─────╨──────┐                    ┌─────╨──────┐     »
«alice: 2/══════════════╡ alice_1=0x1 ╞═══════════════════╡ alice_0=0x1 ╞════»
«                       └────────────┘                    └────────────┘     »
«  bob: 1/═══════════════════════════════════════════════════════════════════»
«                                                                            »
«
«    q_0: ───────────────
«
«    q_1: ───────────────
«          ───────┐ ┌─┐
«    q_2:   End-0 ├─┤M├
«          ───────┘ └╥┘
«alice: 2/═══════════╬═
«                    ║
«  bob: 1/═══════════╩═
«                    0
Measurement counts: {'1 10': 520, '1 00': 566, '1 11': 524, '0 11': 1506, '0 10': 1520, '0 00': 1491, '0 01': 1538, '1 01': 527}
Bob's counts: {'0': 6055, '1': 2137}
Probability of 0: 0.7391357421875
Probability of 1: 0.2608642578125
Expected probability of 1: 0.24999999999999994
```

Program 8: Repeat Program 7 with Bob's corrections removed. Show that Bob's qubit now measures 0 and 1 with equal probability, and explain why the classical bits are necessary.

OUTPUT:



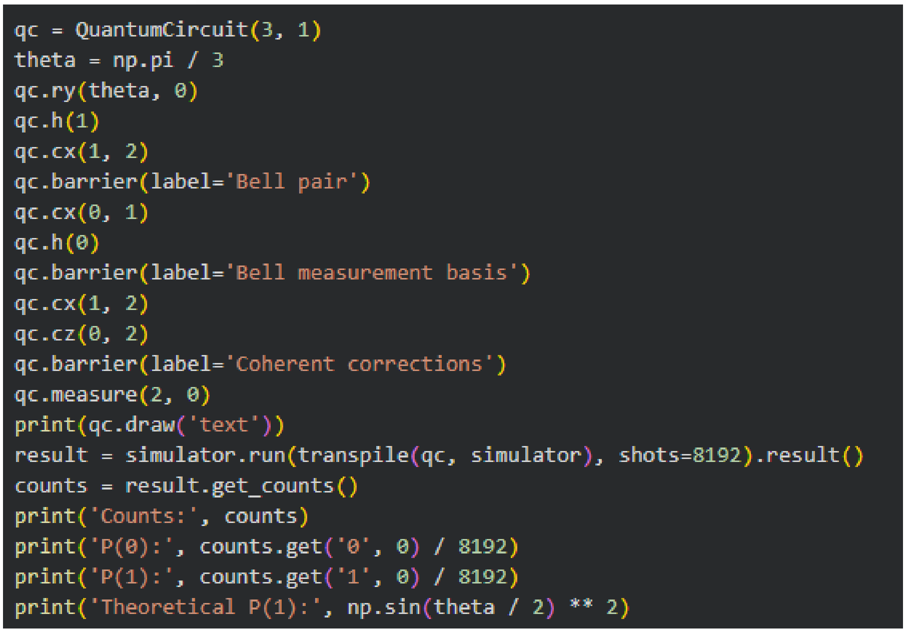
```python
alice = ClassicalRegister(2, 'alice')
bob = ClassicalRegister(1, 'bob')

qc = QuantumCircuit(3)
qc.add_register(alice)
qc.add_register(bob)

theta = np.pi / 3
qc.ry(theta, 0)

teleport_setup(qc, alice)

qc.measure(2, bob[0])

print(qc.draw('text'))

result = simulator.run(transpile(qc, simulator), shots=8192).result()
counts = result.get_counts()

bob_counts = {'0': 0, '1': 0}

for key, value in counts.items():
    bob_bit = key.split()[0]
    bob_counts[bob_bit] += value

print("Measurement counts:", counts)
print("Bob's counts:", bob_counts)
print("Probability of 0:", bob_counts['0'] / 8192)
print("Probability of 1:", bob_counts['1'] / 8192)
```



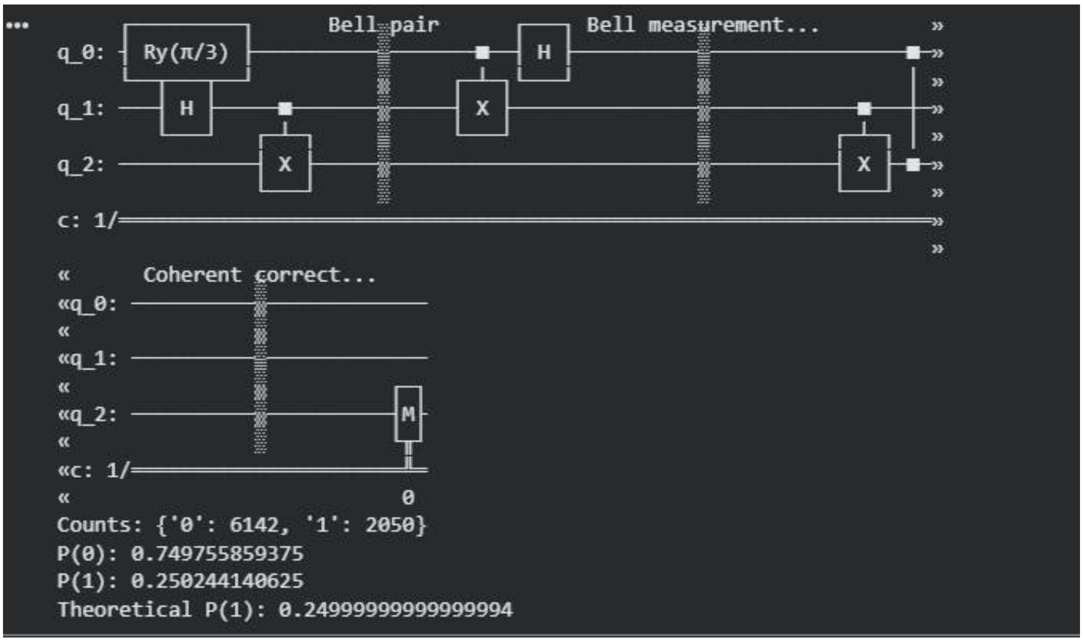
```
         ┌─────────┐           Bell pair         ┌───┐ Alice measuremen... ┌─┐    »
   q_0: ┤ Ry(π/3) ├────────────────░─────────■──┤ H ├─────────░──────────┤M├────»
        └──┬───┬──┘                ░       ┌─┴─┐└───┘         ░          └╥┘┌─┐ »
   q_1: ───┤ H ├──────■────────────░───────┤ X ├──────────────░───────────╫─┤M├─»
           └───┘    ┌─┴─┐          ░       └───┘              ░           ║ └╥┘ »
   q_2: ────────────┤ X ├──────────░──────────────────────────░───────────╫──╫──»
                    └───┘          ░                          ░           ║  ║  »
alice: 2/═════════════════════════════════════════════════════════════════╩══╩══»
                                                                          0  1 »
  bob: 1/═══════════════════════════════════════════════════════════════════════»
                                                                               »
«          Corrections
«    q_0: ─────░──────────
«              ░
«    q_1: ─────░──────────
«              ░      ┌─┐
«    q_2: ─────░──────┤M├─
«              ░      └╥┘
«alice: 2/═════════════╬══
«                      ║
«  bob: 1/═════════════╩══
«                      0
Measurement counts: {'0 10': 488, '0 00': 1640, '1 01': 488, '1 00': 510, '0 11': 499, '1 11': 1466, '1 10': 1498, '0 01': 1603}
Bob's counts: {'0': 4230, '1': 3962}
Probability of 0: 0.516357421875
Probability of 1: 0.483642578125
```

Program 9: Replace the classically controlled corrections with a CNOT gate and a CZ gate controlled by Alice's qubits (deferred measurement). Show that Bob's result is the same as in Program 7.

OUTPUT:



```
qc = QuantumCircuit(3, 1)
theta = np.pi / 3
qc.ry(theta, 0)
qc.h(1)
qc.cx(1, 2)
qc.barrier(label='Bell pair')
qc.cx(0, 1)
qc.h(0)
qc.barrier(label='Bell measurement basis')
qc.cx(1, 2)
qc.cz(0, 2)
qc.barrier(label='Coherent corrections')
qc.measure(2, 0)
print(qc.draw('text'))
result = simulator.run(transpile(qc, simulator), shots=8192).result()
counts = result.get_counts()
print('Counts:', counts)
print('P(0):', counts.get('0', 0) / 8192)
print('P(1):', counts.get('1', 0) / 8192)
print('Theoretical P(1):', np.sin(theta / 2) ** 2)
```



```
     ┌─────────┐        Bell pair      ┌───┐ Bell measurement...        »
q_0: ┤ Ry(π/3) ├────────────░───────■──┤ H ├──────────░────────────────■─»
     └──┬───┬──┘            ░     ┌─┴─┐└───┘          ░                │ »
q_1: ───┤ H ├──────■────────░─────┤ X ├───────────────░──────────■─────┼─»
        └───┘    ┌─┴─┐      ░     └───┘               ░        ┌─┴─┐   │ »
q_2: ────────────┤ X ├──────░─────────────────────────░────────┤ X ├───■─»
                 └───┘      ░                         ░        └───┘     »
c: 1/════════════════════════════════════════════════════════════════════»
                                                                         »
«      Coherent correct...
«q_0: ──────────░──────────────
«               ░
«q_1: ──────────░──────────────
«               ░          ┌─┐
«q_2: ──────────░──────────┤M├
«               ░          └╥┘
«c: 1/══════════════════════╩═
«                           0
Counts: {'0': 6142, '1': 2050}
P(0): 0.749755859375
P(1): 0.250244140625
Theoretical P(1): 0.24999999999999994
```

VIVA Q&A:

1. What is a Bell pair, and how is it prepared?

A Bell pair is a pair of entangled qubits. It is prepared by applying a Hadamard gate to the first qubit, followed by a CNOT gate.

2. How many classical bits does one qubit carry in superdense coding, and what must be shared in advance?

One transmitted qubit can communicate two classical bits. Alice and Bob must share an entangled Bell pair in advance.

3. Which gate does Alice apply for each of the four messages 00, 01, 10 and 11?

- `00` – Identity gate (no operation)
- `01` – X gate
- `10` – Z gate
- `11` – X and Z gates

4. How does Bob decode the message in superdense coding?

Bob applies a CNOT gate followed by a Hadamard gate, then measures both qubits to recover the two-bit message.

5. What is transferred in quantum teleportation: the qubit itself or its state?

The quantum state is transferred, not the physical qubit itself.

6. Why are two classical bits needed for teleportation?

The two classical bits communicate Alice's measurement results to Bob, allowing him to apply the correct quantum operations.

7. Why does teleportation not violate the no-cloning theorem?

The original quantum state is destroyed by Alice's measurement, so teleportation does not create a copy of the original state.

8. Why does teleportation not allow communication faster than light?

Bob needs Alice's classical measurement results, which cannot travel faster than light.

9. Which correction does Bob apply for each of Alice's four measurement results?

Alice's result	**Bob's correction**
`00`	No correction (I)

`01`	X gate
`10`	Z gate
`11`	X and Z gates

10. How are superdense coding and teleportation related to each other?

Both use entanglement and classical information. Superdense coding sends two classical bits using one transmitted qubit, while quantum teleportation transfers an unknown quantum state using two classical bits and a shared entangled pair.

RESULT:

Thus, Bell pair preparation, superdense coding, and quantum teleportation were studied and implemented using quantum circuits. The expected measurement outcomes and the role of entanglement, classical communication, and quantum correction gates were verified through simulation.

EVALUATION (for faculty use):

Evaluation method	Prep	Program	Output	Inference	Viva	Timely	Total
Maximum marks	2	5	3	2	2	1	15
Marks awarded							

Faculty signature with date: